

WEEK 2: Product and Category Management System

- ATCHAYA S [E25AI106]

Scenario:

The customized phone case business wants to maintain information about **phone cases and their customization details**. Each phone case belongs to a specific category or phone model, and the system should manage product details such as **name, price, and stock availability**.

1. Category and Product Tables

Category table:

```
SQL> CREATE TABLE Category(Category_ID VARCHAR2(8),Category_Name VARCHAR2(15), Description VARCHAR2(50));
Table created.
```

Product table:

```
SQL> CREATE TABLE Product(Product_ID VARCHAR2(6),Product_Name VARCHAR2(15), Category_ID VARCHAR2(6), Price NUMBER(9,2), Stock_Quantity Number);
Table created.
```

```
SQL> desc Category;
Name                                     Null?      Type
-----
CATEGORY_ID                             VARCHAR2(8)
CATEGORY_NAME                           VARCHAR2(15)
DESCRIPTION                              VARCHAR2(50)

SQL> desc Product;
Name                                     Null?      Type
-----
PRODUCT_ID                              VARCHAR2(6)
PRODUCT_NAME                            VARCHAR2(15)
CATEGORY_ID                              VARCHAR2(6)
PRICE                                    NUMBER(9,2)
STOCK_QUANTITY                           NUMBER
```

Category and product Tables created.

Now, we have to find the relationship between the tables and apply constraints to the tables.

2. Define Relationships and Constraints

Relationship:

One Category can have many Products.

The relationship between Category and Product is One-to-Many (1 → *).

We identified that Category and Product have a relationship. Now, we need to find which key is used to connect these two tables.

Key Constraints:

PRIMARY KEY

A **Primary Key** is a column that uniquely identifies each row in a table.

In the **Category table**, Category_ID is unique, so it is the Primary Key.

In the **Product table**, Product_ID is unique, so it is the Primary Key.

FOREIGN KEY

A **Foreign Key** is a key that connects two tables.

Here, **Category_ID** from the Category table is used as a Foreign Key in the Product table.

NOT NULL

NOT NULL means a column cannot have an empty (NULL) value.

For example, the Product Name or Category Name should not be empty.

CHECK

CHECK is used to make sure that the value follows a given condition.

For example, the Price of a product should always be greater than 0.

UNIQUE

UNIQUE ensures that duplicate values are not allowed in a column.

For example, if **Category_Name** has a UNIQUE constraint, two categories cannot have the same name.

Defining Relationship and Creating Constraints for Product and Category:

Primary key:

```
SQL> ALTER TABLE Category ADD CONSTRAINT PK_Category PRIMARY KEY (category_ID);  
Table altered.  
  
SQL> ALTER TABLE Product ADD CONSTRAINT PK_Product PRIMARY KEY (product_ID);  
Table altered.
```

Creating relationship with using FOREIGN KEY:

```
SQL> ALTER TABLE Product ADD CONSTRAINT FK_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID);  
Table altered.
```

Adding Constraints:

NOT NULL:

```
SQL> ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;  
Table altered.  
SQL> ALTER TABLE Product MODIFY Product_Name VARCHAR(100) NOT NULL;  
Table altered.
```

CHECK:

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Price CHECK (Price > 0);  
Table altered.  
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);  
Table altered.
```

UNIQUE:

```
SQL> ALTER TABLE Category ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);  
Table altered.
```

3. INSERTING VALUES INTO TABLES

Category Table:

```
SQL> INSERT INTO Category
  2  VALUES(
  3  '&Category_ID',
  4  '&Category_Name',
  5  '&Description');
Enter value for category_id: CAT00001
old   3: '&Category_ID',
new   3: 'CAT00001',
Enter value for category_name: Electronics
old   4: '&Category_Name',
new   4: 'Electronics',
Enter value for description: Electronic Gadgets and Devices
old   5: '&Description')
new   5: 'Electronic Gadgets and Devices')

1 row created.

SQL> /
Enter value for category_id: CAT00002
old   3: '&Category_ID',
new   3: 'CAT00002',
Enter value for category_name: Clothing
old   4: '&Category_Name',
new   4: 'Clothing',
Enter value for description: Men , Women and Kids Apparels
old   5: '&Description')
new   5: 'Men , Women and Kids Apparels')

1 row created.

SQL> /
Enter value for category_id: CAT00003
old   3: '&Category_ID',
new   3: 'CAT00003',
Enter value for category_name: Stationary
old   4: '&Category_Name',
new   4: 'Stationary',
Enter value for description: Notebooks, pens and Papers
old   5: '&Description')
new   5: 'Notebooks, pens and Papers')
```

```

SQL> INSERT INTO CATEGORY
  2  VALUES(
  3  '&Category_ID',
  4  '&Category_Name',
  5  '&Description');
Enter value for category_id: CAT00004
old   3: '&Category_ID',
new   3: 'CAT00004',
Enter value for category_name: Home Appliances
old   4: '&Category_Name',
new   4: 'Home Appliances',
Enter value for description: Home essentials and needs
old   5: '&Description')
new   5: 'Home essentials and needs')

1 row created.

SQL> /
Enter value for category_id: CAT00005
old   3: '&Category_ID',
new   3: 'CAT00005',
Enter value for category_name: Mobile Parts
old   4: '&Category_Name',
new   4: 'Mobile Parts',
Enter value for description: New Spares for Mobile
old   5: '&Description')
new   5: 'New Spares for Mobile')

1 row created.

SQL> /
Enter value for category_id: CAT00006
old   3: '&Category_ID',
new   3: 'CAT00006',
Enter value for category_name: Provisions
old   4: '&Category_Name',
new   4: 'Provisions',
Enter value for description: Kitchen Needs
old   5: '&Description')
new   5: 'Kitchen Needs')

```

Product Table:

1 row created.

```
SQL> /
Enter value for product_id: P10002
old   3: '&Product_ID',
new   3: 'P10002',
Enter value for product_name: Croptops and Shirt
old   4: '&Product_Name',
new   4: 'Croptops and Shirt',
Enter value for categoryid: CAT00002
old   5: '&CategoryID',
new   5: 'CAT00002',
Enter value for price: 2000
old   6: &Price,
new   6: 2000,
Enter value for stock_quantity: 35
old   7: &Stock_Quantity)
new   7: 35)
```

1 row created.

```
SQL> /
Enter value for product_id: P10003
old   3: '&Product_ID',
new   3: 'P10003',
Enter value for product_name: Unruled Note
old   4: '&Product_Name',
new   4: 'Unruled Note ',
Enter value for categoryid: CAT00003
old   5: '&CategoryID',
new   5: 'CAT00003',
Enter value for price: 50
old   6: &Price,
new   6: 50,
Enter value for stock_quantity: 55
old   7: &Stock_Quantity)
new   7: 55)
```

1 row created.

```
SQL> INSERT INTO Product
  2  VALUES(
  3  '&Product_ID',
  4  '&Product_Name',
  5  '&Category_ID',
  6  &Price,
  7  &Stock_Quantity);
Enter value for product_id: P1004
old   3: '&Product_ID',
new   3: 'P1004',
Enter value for product_name: Television
old   4: '&Product_Name',
new   4: 'Television',
Enter value for category_id: CAT00004
old   5: '&Category_ID',
new   5: 'CAT00004',
Enter value for price: 15000
old   6: &Price,
new   6: 15000,
Enter value for stock_quantity: 5
old   7: &Stock_Quantity)
new   7: 5)
```

1 row created.

```
SQL> /
Enter value for product_id: P1005
old   3: '&Product_ID',
new   3: 'P1005',
Enter value for product_name: Speaker
old   4: '&Product_Name',
new   4: 'Speaker',
Enter value for category_id: CAT00004
old   5: '&Category_ID',
new   5: 'CAT00004',
Enter value for price: 13000
old   6: &Price,
new   6: 13000,
Enter value for stock_quantity: 6
old   7: &Stock_Quantity)
```

```
1 row created.

SQL> /
Enter value for product_id: P1006
old 3: '&Product_ID',
new 3: 'P1006',
Enter value for product_name: Camera
old 4: '&Product_Name',
new 4: 'Camera',
Enter value for category_id: CAT00005
old 5: '&Category_ID',
new 5: 'CAT00005',
Enter value for price: 4500
old 6: &Price,
new 6: 4500,
Enter value for stock_quantity: 16
old 7: &Stock_Quantity)
new 7: 16)

1 row created.

SQL> /
Enter value for product_id: P1007
old 3: '&Product_ID',
new 3: 'P1007',
Enter value for product_name: Charging pin
old 4: '&Product_Name',
new 4: 'Charging pin',
Enter value for category_id: CAT00005
old 5: '&Category_ID',
new 5: 'CAT00005',
Enter value for price: 500
old 6: &Price,
new 6: 500,
Enter value for stock_quantity: 45
old 7: &Stock_Quantity)
new 7: 45)

1 row created.
```

After Insertion Category Table:

```

CATEGORY
-----
CATEGORY_NAME
-----
DESCRIPTION
-----

CAT00003
Stationary
Notebooks, pens and Papers

CAT00004
Home Appliances

CATEGORY
-----
CATEGORY_NAME
-----
DESCRIPTION
-----
Home essentials and needs

CAT00005
Mobile Parts
New Spares for Mobile

CAT00006

CATEGORY
-----
CATEGORY_NAME
-----
DESCRIPTION
-----
Provisions
Kitchen Needs

6 rows selected.

```

After Insertion Product Table:

```

PRODUC
-----
PRODUCT_NAME
-----
CATEGORY      PRICE  STOCK_QUANTITY
-----
Camera
CAT000005      4500           16

P1007
Charging pin
CAT000005      500           45

PRODUC
-----
PRODUCT_NAME
-----
CATEGORY      PRICE  STOCK_QUANTITY
-----
P1008
Species
CAT000006      100           60

P1009
Utencils
CAT000006     1000           55

PRODUC
-----
PRODUCT_NAME
-----
CATEGORY      PRICE  STOCK_QUANTITY
-----

9 rows selected.

```

CRUD Operations:

Insertion record

```
SQL> INSERT INTO Product (Product_ID, Product_Name, Category_ID, Price, Stock_Quantity)
  2  VALUES ('P10004', 'Tablet', 'CAT00002', 10000, 10);

1 row created.
```

Update Price

```
SQL> UPDATE Product SET Price = 76000 WHERE Product_Name = 'Laptops';

1 row updated.
```

Deletion

```
SQL> DELETE FROM Product WHERE Product_Name = 'Laptops';

1 row deleted.
```

4. Category-Wise Product Reports

SQL query:

```
SELECT c.CATEGORY_ID,
       c.CATEGORY_NAME,
       p.PRODUCT_ID,
       p.PRODUCT_NAME,
       p.PRICE,
       p.STOCK_QUANTITY
FROM Category c
JOIN Product p
ON c.CATEGORY_ID = p.CATEGORY_ID
ORDER BY c.CATEGORY_ID;
```

```

SQL> SELECT c.CATEGORY_ID,
2         c.CATEGORY_NAME,
3         p.PRODUCT_ID,
4         p.PRODUCT_NAME,
5         p.PRICE,
6         p.STOCK_QUANTITY
7 FROM Category c
8 JOIN Product p
9 ON c.CATEGORY_ID = p.CATEGORY_ID
10 ORDER BY c.CATEGORY_ID;

```

```

CATEGORY
-----
CATEGORY_NAME
-----
PRODUC
-----
PRODUCT_NAME
-----
PRICE STOCK_QUANTITY
-----
CAT00002
Clothing
P10002

CATEGORY
-----
CATEGORY_NAME
-----
PRODUC
-----
PRODUCT_NAME
-----
PRICE STOCK_QUANTITY
-----
Croptops and Shirt
2000 35

```

```

PRODUC
-----
PRODUCT_NAME
-----
PRICE STOCK_QUANTITY
-----
Species 100 60

CATEGORY
-----
CATEGORY_NAME
-----
PRODUC
-----
PRODUCT_NAME
-----
PRICE STOCK_QUANTITY
-----
CAT00006
Provisions
P1009

CATEGORY
-----
CATEGORY_NAME
-----
PRODUC
-----
PRODUCT_NAME
-----
PRICE STOCK_QUANTITY
-----
Utencils 1000 55

9 rows selected.

```

Counting Number of products in each Category:

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count
2 FROM Product p
3 JOIN Category c ON p.Category_ID = c.Category_ID
4 GROUP BY c.Category_Name;
```

```
CATEGORY_NAME
-----
PRODUCT_COUNT
-----
Clothing      2

Stationary    1

Home Appliances 2

CATEGORY_NAME
-----
PRODUCT_COUNT
-----
Mobile Parts  2

Provisions    2
```

Finding highest priced product in each category:

```
8 );
```

```
CATEGORY_NAME
-----
PRODUCT_NAME
-----
HIGHEST_PRICE
-----
Stationary
Unruled Note    50

Clothing
Tablet          10000

CATEGORY_NAME
-----
PRODUCT_NAME
-----
HIGHEST_PRICE
-----
Home Appliances
Television      15000

Mobile Parts
Camera

CATEGORY_NAME
-----
PRODUCT_NAME
-----
HIGHEST_PRICE
-----
4500

Provisions
Utencils        1000
```

Display categories having more than 5 products

```
SQL> SELECT c.Category_Name
2  FROM Product p
3  JOIN Category c ON p.Category_ID = c.Category_ID
4  GROUP BY c.Category_Name
5  HAVING COUNT(p.Product_ID) > 5;

no rows selected
```

No rows selected because all categories having less than 5 products.

Find average product price category-wise

```
SQL> SELECT c.Category_Name, AVG(p.Price) AS Avg_Price
2  FROM Product p
3  JOIN Category c ON p.Category_ID = c.Category_ID
4  GROUP BY c.Category_Name;
```

CATEGORY_NAME

AVG_PRICE

Clothing
6000

Stationary
50

Home Appliances
14000

CATEGORY_NAME

AVG_PRICE

Mobile Parts
2500

Provisions
550

